Deal with Devil

Frontend Build Spec

Participant app and admin panel for the Hacker Auction event

Oct 20, 2026 | @Vishesh
Backend: Spring Boot 4.1.1 (Java 21) and PostgreSQL, finished and fixed
Frontend: React, two apps behind one login

> **What this is.** One self-contained brief for the frontend developer: every screen, the API call behind each element, every rule and every edge case, an event-day walkthrough, the full API list, all data types, and a build and test plan. The repository docs (`docs/frontend-api-contract.md`) remain the endpoint reference.

Contents

1. What we are building 4

2. Core concepts the UI must explain 5

3. Global foundations 6

4. Participant app 8

4.1 Login . . . 8

4.2 Profile form (first login) . . . 8

4.3 Dashboard . . . 8

4.4 Round preview (problem cards) . . . 8

4.5 Sealed bid screen . . . 9

4.6 My results and my problems . . . 9

4.7 Transaction history . . . 9

4.8 Leaderboard . . . 9

4.9 Rules page . . . 10

4.10 Pop-ups and the notification bell . . . 10

5. Admin panel 10

5.1 Round and auction controls . . . 10

5.2 Problem bank . . . 11

5.3 Participant manager . . . 11

5.4 Wallet and devil buttons . . . 12

5.5 Ledger viewer and undo . . . 12

5.6 Audit log . . . 13

6. Phase by phase behaviour 13

7. Pop-ups and notifications 14

7.1 The event feed . . . 14

7.2 What to do with each event . . . 14

7.3 The eleven event types . . . 15

7.4 Pop-up behaviour . . . 15

7.5 The notification bell . . . 15

8. A full event day, in order 16

8.1 Before the day . . . 16

8.2 Opening the event . . . 16

8.3 One round . . . 16

8.4 When things go wrong on the day . . . 17

9. Edge cases and what the UI does 17

10. Full API reference 19

10.1 Public . . . 19

10.2 Any signed-in user . . . 19

10.3 Admin: rounds and auction . . . 19

10.4 Admin: problem bank . . . 20
10.5 Admin: participants . . . 20
10.6 Admin: wallet, ledger and audit . . . 20
11. Data types **21**
11.1 Enums . . . 21
11.2 Errors, auth, config and profile . . . 21
11.3 Ledger and events . . . 22
11.4 Rounds, problems and bids . . . 22
11.5 Admin wallet and audit . . . 23
12. Design guidance, decisions, build order and tests **23**
12.1 Design guidance . . . 23
12.2 Open decisions . . . 23
12.3 Suggested build order . . . 24
12.4 Mock data and test plan . . . 24
12.5 Definition of done and handover . . . 25

1. What we are building

Two React apps, a participant app and an admin panel, that sit on top of a finished Spring Boot backend and run a live bidding event for individual participants. This document is everything a frontend developer needs: every screen, the API call behind each element, every rule, and every edge case. The backend is done and fixed; build against it exactly as written here.

The event in one paragraph. Participants spend virtual coins to bid, in secret, on tech problems that are later solved on HackerRank. Winners of a problem pay the price set by the next-best bid (a Vickrey, or second-price, auction). Each participant also has a corruption level from 1 to 3 that decides which version of a problem's hint they get, and the admin can trade corruption for coins (the devil's deal). The app never hosts the solving: it handles login, coins, bidding, results, hints, pop-ups and the leaderboard. Admins run the whole event from the admin panel.

The two apps.

App	Who	What it is for
Participant app	Role PARTICIPANT	See the round, read the question previews, place sealed bids, see results and the HackerRank link of won problems, watch coins and corruption level, check the leaderboard
Admin panel	Role ADMIN	Run rounds and phases, manage problems and hints, search participants, credit and deduct coins, run the devil's deal, undo mistakes, read the ledger and audit log

Both can live in one React project. The login response contains `role`; route a PARTICIPANT to the participant app and an ADMIN to the admin panel. The server rejects admin calls from participants on its own, so the routing is for usability, not security.

Suggested stack (the backend does not care, these are recommendations): React with TypeScript and Vite, React Router, a data-fetching library with polling support such as TanStack Query, Google Identity Services for sign-in, and MSW or JSON fixtures to build screens before the backend runs (section 12).

How to read this document.

- Sections 2 and 3 explain the concepts and the shared plumbing every screen needs.
- Sections 4 and 5 are the screen-by-screen specification for the two apps.
- Section 6 gives behaviour per round phase, and section 7 the pop-up rules.
- Section 8 walks through a real event day in order, so you can see the screens used together.
- Sections 9 to 11 are reference: edge cases, the full API list and the data types.
- Section 12 covers design guidance, open decisions, a build order and a test plan.

The endpoint list and every request and response shape also exist in `docs/frontend-api-contract.md` in the repository; this document adds the screens, flows and rules around them.

> **Backend status.** All 180 backend tests pass and the app starts normally. A real Google sign-in has not been tried yet, so expect to test the login flow early.

2. Core concepts the UI must explain

Participants will only bid well if the interface makes eight ideas obvious: coins, the sealed second-price auction, the reserve price, slots, rounds and phases, corruption level, the devil, and hints. Put short plain-language versions of these on the Rules page (section 4.9) and in tooltips next to the numbers they explain.

Concept	What it means	Where the UI must show it
Coins	Whole numbers only, never fractions. Everything costs or pays coins. The config has no automatic starting grant, so the organisers decide how people get their first coins, most likely an admin credit	Dashboard header (always visible), bid screen (balance left after bids), history, leaderboard
Sealed bid	One secret bid per participant per problem, editable until bidding closes. Nobody ever sees another person's bid	Bid screen: the participant's own amount only
Slots	A problem can have several winners. `slots` is how many	Problem card: "3 winners"
Reserve price	Minimum bid, also called `basePrice`. A bid below it is rejected	Problem card and bid input (disable or warn under it)
Vickrey price	The top `slots` bidders win, and every winner pays the same price: the (slots+1)th highest bid, or the reserve price if there are not enough bids. Winners never pay their own bid	Rules page, result screen ("you bid 150, you paid 100")
Ties	Equal bids: the earlier submission wins. Changing a bid amount resets its submission time	Rules page
Round	One auction cycle with its own set of problems. Rounds are numbered 1, 2, 3 and only the latest is current	Dashboard, round banner
Phase	Where the current round is: PREVIEW, BIDDING, SETTLEMENT, SOLVING, RESULTS. Only the admin moves it, and only forward	Phase banner on every participant screen (section 6)
Corruption level	A whole number from 1 to 3, visible to the participant. It decides which hint version they get	Dashboard badge, pop-ups on change
Devil's deal	Admin action: corruption level +1 and the participant receives coins (`devil.dealPayout`)	Pop-up DEVIL_DEAL
Redemption	Admin action: corruption level -1 and the participant pays coins (`devil.redemptionCost`, always higher than the deal payout so looping cannot farm coins)	Pop-up REDEMPTION
Hint	Each problem has three hint versions, one per corruption level, each with a password. The admin looks up the right one for a participant and hands it out; hints cost the same for every problem (`hintPrice`). Participants never see hint text in the app	Admin hint lookup only
Ledger	Append-only list of every coin change with reason and balance after. Undo adds a reversing entry, never deletes	Participant history, admin ledger viewer
Leaderboard	Participants ranked by coins, equal coins share a rank	Leaderboard screen

Worked example for the Rules page. A problem has 2 slots and a reserve price of 20. Four people bid 90, 70, 50 and 30. The two highest bidders (90 and 70) win. Both pay 50, the third-highest bid. The 90 bidder saves 40 coins and the 70 bidder saves 20. If only two people had bid, there is no third bid, so both winners would pay the reserve price of 20.

What a participant must understand about bidding.

- Bid what the problem is really worth to you. Your own bid decides whether you win; someone else's bid sets the price.
- All of your bids in a round together cannot add up to more than your coins. Coins are only taken at settlement, but you cannot promise more than you have.
- If an admin deducts coins after you bid and you can no longer pay at settlement, that bid is marked UNAFFORDABLE. You pay nothing, and the slot is not given to anyone else.
- The HackerRank link of a problem is only revealed to people who won it.

3. Global foundations

Build these once, before any screen: the API client, the session, the role router, the profile gate, the polling loops and one shared error handler. Every screen in sections 4 and 5 relies on them.

API basics.

Thing	Value
Base URL	`VITE_API_URL`, development default `http://localhost:8080`. All routes start with `/api`
Format	JSON in and out. Send `Content-Type: application/json` on POST and PUT bodies
Auth header	`Authorization: Bearer <JWT>` on every request except `GET /api/health`, `POST /api/auth/google` and `POST /api/auth/admin-fallback`
CORS	Development origins `http://localhost:5173` (Vite) and `http://localhost:3000`. The production React origin must be added on the backend before launch
Numbers	Coins are whole numbers. IDs are numbers
Timestamps	ISO-8601 UTC strings such as `2026-11-02T10:30:00Z`; show them in the participant's local time
Enums	Uppercase strings (section 11)
Missing values	`null`

One API client. Wrap `fetch` once: add the base URL and the Bearer header, parse JSON, return nothing for status 204, and on any non-2xx response throw an `ApiError` carrying status, code, message and details. Every screen uses this client, and every screen shows `message` to the user. A reference implementation is in `docs/frontend-api-contract.md` section 9.

Sign-in and session

- The login screen shows the Google button. Google Identity Services returns an ID token in the browser.
- Send it to `POST /api/auth/google` with `{ idToken }`. The response is `{ token, expiresAt, profileComplete, role }`.
- Keep the token in memory or `sessionStorage`, never in `localStorage` and never in a cookie the page script cannot control.
- Route by role: PARTICIPANT goes to the participant app, ADMIN to the admin panel. A participant with `profileComplete: false` goes to the profile form first.
- The token lasts about 3 hours. When fewer than 15 minutes remain before `expiresAt`, call `POST /api/auth/refresh` (no body) and replace the token silently. A long event therefore never needs a second Google sign-in.
- On TOKEN_EXPIRED, TOKEN_INVALID or UNAUTHORIZED (all 401), clear the token and return to the login screen. TOKEN_INVALID also appears when an admin signs someone out everywhere.
- Backup admin sign-in: a small "Admin fallback sign-in" link on the login screen opens an email and passcode form for `POST /api/auth/admin-fallback`. It only works for allowlisted admin emails, and after five wrong tries it answers TOO_MANY_REQUESTS (429) for that email for a few minutes.

Profile gate. Until a participant saves the first-login form, the server answers 403 PROFILE_INCOMPLETE for everything except `GET /api/me`, `PUT /api/me/profile`, `GET /api/config` and the `/api/auth/*` calls. The router must therefore keep an incomplete participant on the profile form. Admins are not gated.

Fetch once after login. `GET /api/config` (prices, rewards, hint price, devil rates, corruption level limits) and `GET /api/me`. Keep both in global state; many popups and labels use the config values.

Polling. There are no WebSockets. Poll while the relevant screen is mounted and pause polling when the browser tab is hidden.

What	Endpoint	Interval
Round, phase, pause, countdown	`GET /api/rounds/current`	5 seconds
Pop-ups and notifications	`GET /api/me/events?after=<lastId>`	5 seconds

What	Endpoint	Interval
Coins and corruption level	`GET /api/me`	After every event that arrives and every 15 seconds
Leaderboard	`GET /api/leaderboard`	15 seconds, only while the screen is open

Server clock. The countdown to the bidding deadline must not trust the participant's clock. Every `GET /api/rounds/current` returns `serverTime`. Store `offset = serverTime - Date.now()` at fetch time and count down to `biddingDeadline` using `Date.now() + offset`. When the countdown reaches zero, show "Bidding closed" immediately and refetch the round; the server is the final judge and rejects late bids with BIDDING_CLOSED.

Role routing. Two route trees behind one login. Never render admin routes for a participant, and never let an admin place bids (the server answers 403 to an admin bidding). A 403 FORBIDDEN on a route the user should not reach means a routing bug; show a generic "You are not allowed to do that".

Errors. Every error has the shape `{ timestamp, status, code, message, path, details? }`. Switch on `code`, show `message`, and put each entry of `details` next to the form field it names. One global handler decides what to do:

code	HTTP	What the UI does
VALIDATION_FAILED	400	Show details under the matching fields
MALFORMED_REQUEST, INVALID_PARAMETER	400	Generic "something went wrong"
INVALID_AMOUNT	400	Show the message
UNAUTHORIZED, TOKEN_EXPIRED, TOKEN_INVALID	401	Clear the token, go to login
INVALID_GOOGLE_TOKEN, INVALID_CREDENTIALS	401	"Sign-in failed, try again"
FORBIDDEN	403	"You are not allowed to do that"
EMAIL_NOT_VERIFIED	403	"Verify your Google email first"
PROFILE_INCOMPLETE	403	Go to the profile form
NOT_FOUND, USER_NOT_FOUND, PROBLEM_NOT_FOUND	404	Show the message
ROUND_NOT_FOUND	404	Normal before round 1: show "The event has not started yet", not an error
METHOD_NOT_ALLOWED / UNSUPPORTED_MEDIA_TYPE	405 / 415	Generic message (a client bug)
CONFLICT	409	Show the message. Also used for "HackerRank ID already taken" (details.hackerRankId) and "already undone"
CONCURRENT_UPDATE	409	Refetch the data and let the user retry
INSUFFICIENT_BALANCE	409	Show the message
BID_EXCEEDS_BALANCE	409	"Your bids add up to more coins than you have"
INVALID_PHASE	409	Show the message and refetch the round
BIDDING_CLOSED	409	Lock the bid screen and refetch the round
ALREADY_SETTLED	409	Show the message and refetch the round
CORRUPTION_LIMIT_REACHED	409	Show the message
TOO_MANY_REQUESTS	429	"Slow down"
SERVICE_UNAVAILABLE	503	"Try again shortly"; never sign the user out
INTERNAL_ERROR	500	Generic message

Security rules for the frontend. The JWT lives in the browser, so the main risk is injected script. Do not paste third-party scripts into the page, avoid `dangerouslySetInnerHTML`, and render the admin-written question preview as plain text (it is plain text by design). Hint passwords and HackerRank links only ever come from admin endpoints or from a won bid;

never cache them in places other users could reach.

4. Participant app

Ten screens plus one global layer (the pop-ups). Participants will mostly be on phones during the event, so design mobile first. Every screen below lists its purpose, what it shows, the API calls behind it, and the rules that matter.

Global layout. A fixed top bar shows the participant's name, coin balance and corruption level badge on every screen, plus the notification bell. Navigation items: Dashboard, Round (preview and bidding), My problems, History, Leaderboard, Rules, Profile. A phase banner (section 6) sits under the top bar whenever a round exists. The pop-up layer (4.10) is mounted once at the top of the app so it works on every screen.

4.1 Login

- **Purpose:** sign in with Google and enter the right app.
- **Shows:** the Google sign-in button, a small "Admin fallback sign-in" link.
- **Calls:** `POST /api/auth/google` with `{ idToken }`, then `GET /api/config` and `GET /api/me`. The fallback form calls `POST /api/auth/admin-fallback` with `{ email, passcode }`.
- **Rules:** route by role and `profileComplete` (section 3). Show INVALID_GOOGLE_TOKEN and EMAIL_NOT_VERIFIED messages on this screen. While the request runs, disable the button and show a spinner.

4.2 Profile form (first login)

- **Purpose:** mandatory one-time form; the app is locked until it is saved.
- **Fields:** Name (required, up to 100 characters) and HackerRank ID (required, up to 50 characters, no spaces, unique ignoring letter case). Prefill from `GET /api/me`.
- **Calls:** `PUT /api/me/profile` with `{ name, hackerRankId }`, which returns the updated MeResponse.
- **Errors:** VALIDATION_FAILED puts `details.name` and `details.hackerRankId` under their fields. CONFLICT with `details.hackerRankId` means that HackerRank ID is taken by someone else; say so next to the field.
- **Rules:** after a successful save go to the dashboard. The same form stays reachable from the Profile menu so a participant can correct their HackerRank ID later; admins use that ID to find people, so it must be right.

4.3 Dashboard

- **Purpose:** the home screen: where am I, what can I do now.
- **Shows:** coin balance, corruption level with a one-line explanation, round number, current phase (large), countdown to the bidding deadline when one is set, a paused banner when the round is paused, and a primary button that matches the phase ("See problems" in preview, "Place bids" in bidding, "See my results" after settlement).
- **Calls:** `GET /api/me` (every 15 seconds and after every pop-up event) and `GET /api/rounds/current` (every 5 seconds).
- **States:** ROUND_NOT_FOUND (404) before round 1 shows "The event has not started yet" with no error styling.

4.4 Round preview (problem cards)

- **Purpose:** let participants study the problems before and during bidding.
- **Shows:** one card per problem with title, category, tier badge (Bronze, Silver, Gold), teaser, the question preview text written by the admin, the number of winners (`slots`) and the reserve price (`basePrice`). Show the question preview in full or behind "Read more", as plain text with line breaks preserved.
- **Calls:** `GET /api/rounds/current/problems`, ordered by id.
- **Rules:** this list is available in every phase, so keep the cards visible while bidding. The HackerRank link is never part of it. An empty array means "Problems will appear here soon". In the PREVIEW phase there is no bid input; show "Bidding opens soon".

4.5 Sealed bid screen

- **Purpose:** place and change secret bids while the phase is BIDDING.
- **Shows:** the same problem cards, each with a bid amount input, a Save button, and the participant's current saved bid. A sticky summary bar shows: coins, total of all bids, coins left after bids, and the deadline countdown.
- **Calls:** `GET /api/rounds/current/problems` and `GET /api/me/bids` to load, `PUT /api/rounds/current/problems/{problemId}/bid` with `{ amount }` to save. The response is the saved bid with status PENDING and `submittedAt`.
- **Client-side checks** (the server repeats them): amount is a whole number, at least 1 and at least the problem's reserve price; the total of all the participant's bids must not exceed their coins. Show the running total live and disable Save when a bid would push the total over the balance.
- **Server errors:** BID_EXCEEDS_BALANCE (total too high), VALIDATION_FAILED with `details.amount` (below the reserve price), BIDDING_CLOSED (lock every input and refetch the round), PROBLEM_NOT_FOUND.
- **Rules:**
  - One bid per problem. Saving again replaces the amount.
  - A bid cannot be withdrawn, only changed to a new amount, and the lowest allowed amount is the reserve price. Tell participants this next to the input.
  - Changing the amount resets the submission time, which matters for ties, so do not encourage needless edits.
  - Only the participant's own bids are ever shown. Never display anyone else's.
  - When the round is paused, or the deadline has passed, show every input as read-only with a banner explaining why. The pause can end, so keep polling.
  - Admins never see this screen; an admin placing a bid gets 403.

4.6 My results and my problems

- **Purpose:** after settlement, show what the participant won, what it cost, and where to solve it.
- **Shows:** a list of the participant's bids with problem title, own bid amount, status (WON, LOST or UNAFFORDABLE), and for winners the price paid and an "Open on HackerRank" button (opens in a new tab with `rel="noopener"`). "My problems" is the same list filtered to WON. Explain the price clearly: "You bid 150 and paid 100".
- **Calls:** `GET /api/me/bids`.
- **Rules by phase:** in SETTLEMENT show "Bidding closed, results coming" (the results exist only after the admin runs settlement). From SOLVING on, show results. `hackerRankLink` is null unless the bid is WON; never construct a link yourself. For UNAFFORDABLE, explain: "You would have won, but you no longer had enough coins, so nothing was charged." The BID_WON, BID_LOST and BID_UNAFFORDABLE pop-ups arrive at the same time as settlement.

4.7 Transaction history

- **Purpose:** an honest statement of every coin movement.
- **Shows:** a table or list, newest first: date and time, type (PRESET_CREDIT, HINT_DEDUCTION, BID_SETTLEMENT, DEVIL_DEAL, REDEMPTION, CUSTOM, REVERSAL), reason text, signed amount (green for credits, red for deductions) and the balance after. A REVERSAL entry points at the entry it undid through `reversalOf`.
- **Calls:** `GET /api/me/history`.
- **States:** empty state "No transactions yet".

4.8 Leaderboard

- **Purpose:** friendly competition by coins.
- **Shows:** rank, name, HackerRank ID and coins for every participant with a completed profile, sorted by coins (then name). Equal coins share a rank, so the sequence can read 1, 1, 3. Highlight the row where `userId` equals the signed-in user.
- **Calls:** `GET /api/leaderboard`, polled every 15 seconds while the screen is open.

4.9 Rules page

- **Purpose:** the participant's reference for how the event works.
- **Shows (static):** the five phases in plain words, how sealed second-price bidding works with the worked example from section 2, the reserve price and tie rule, what the corruption level does, what the devil's deal and redemption are, how hints are given out by the organisers, and the bid limit (all bids together cannot exceed your coins).
- **Calls:** none beyond the config fetched at login. Print the live numbers from `GET /api/config`: base price per tier, solve reward per tier, hint price, devil payout and redemption cost, and the corruption level range. These values are placeholders until the organisers fix them, so never hard-code them.

4.10 Pop-ups and the notification bell

Mounted globally. It polls `GET /api/me/events` and turns personal events into pop-ups and all events into bell entries. The full rules, event types and styling are in section 7.

5. Admin panel

The admin panel is where the event is run, often under time pressure, so the guiding rule is: every action is one clear button, every coin action asks for confirmation, and every result is visible immediately. Admins can use a laptop; design for desktop first.

Global layout. Left navigation: Round and auction, Problems, Participants, Ledger, Audit log. A header shows the current round number, phase and paused state on every admin screen (polled every 5 seconds from `GET /api/rounds/current`). Admins are not gated by the profile form.

Confirm popups. Every call under `/api/admin/wallet/` and the sign-out-everywhere button opens a confirm popup first. The popup names the participant (name and HackerRank ID) and states the exact effect in coins, for example: "Credit 200 coins to Asha Rao (ace_coder) for solving a Silver problem?" with Cancel and Confirm buttons. Take the amounts from `GET /api/config`. Disable the Confirm button while the request is running so a double click cannot send it twice. After success, show a toast with the new balance (`balanceAfter` in the response) and update the participant's row.

5.1 Round and auction controls

- **Purpose:** move the event through its phases and settle the auction.
- **Shows:** the round number, the current phase as a stepper (PREVIEW, BIDDING, SETTLEMENT, SOLVING, RESULTS), paused state, bidding deadline, and one primary button that changes with the phase.
- **Calls:** `GET /api/rounds/current` to read state; `POST /api/admin/rounds` to start a round; `POST /api/admin/rounds/current/advance`, `/pause`, `/resume`; `PUT /api/admin/rounds/current/bidding-deadline` with `{ deadline }`; `POST /api/admin/rounds/current/settle`; `GET /api/admin/rounds/current/settlement`.

Current phase	Primary button	Call	Notes
No round yet	Start round 1	`POST /api/admin/rounds`	Returns the new round in PREVIEW
PREVIEW	Open bidding	`POST .../advance`	Refused with INVALID_PHASE if the round has no problems. Deadline picker enabled
BIDDING	Close bidding	`POST .../advance`	Moves to SETTLEMENT. Deadline picker still enabled
SETTLEMENT	Run settlement	`POST .../settle`	There is no advance in this phase. Charges winners, writes results, moves to SOLVING
SOLVING	Show results	`POST .../advance`	Moves to RESULTS
RESULTS	Start next round	`POST /api/admin/rounds`	Creates round N+1 in PREVIEW

- Pause and resume is always available as a toggle. While the round is paused, disable every phase button, and participants cannot bid.
- **Bidding deadline:** a date and time picker, allowed in PREVIEW or BIDDING. The time must be in the future (VALIDATION_FAILED with `details.deadline` otherwise). Send `null` to clear it. The deadline only blocks late bids; it does not move the phase, so the admin still presses Close bidding.

- After every call refetch `GET /api/rounds/current`. On CONCURRENT_UPDATE (two admins clicked at once) refetch and let the admin retry. Pressing Settlement twice is safe: the second attempt answers ALREADY_SETTLED or CONCURRENT_UPDATE, and coins are charged once.
- **Settlement summary:** after settling, show a table of every problem with bid count, winner count, unaffordable count and clearing price, plus the total coins collected. The same summary can be read again later with `GET /api/admin/rounds/current/settlement` (INVALID_PHASE before settlement).

5.2 Problem bank

- **Purpose:** create the problems of a round, write their question previews and hints, and see who won.
- **Shows:** a list of all problems (optionally filtered by round) with title, tier, slots, reserve price and round. Selecting one opens its editor.
- **Calls:** `GET /api/admin/problems` (`?roundId=` optional), `GET /api/admin/problems/{id}`, `POST /api/admin/problems`, `PUT /api/admin/problems/{id}`, `DELETE /api/admin/problems/{id}`, `PUT /api/admin/problems/{id}/preview`, `PUT /api/admin/problems/{id}/hints/{level}`, `GET /api/admin/problems/{id}/winners`.

Problem form fields.

Field	Rule
Title	Required, up to 150 characters
Category	Required, up to 60 characters
Tier	Required: Bronze, Silver or Gold
Slots	Required whole number from 1 to 100 (number of winners)
HackerRank link	Required, up to 500 characters, must start with http:// or https://
Reserve price (basePrice)	Optional, 0 or more. Leave empty to use the tier default from `GET /api/config`
Teaser	Optional, up to 500 characters; a one-line tease shown on the card
Round	Optional; empty keeps the problem in the bank only

- `PUT /api/admin/problems/{id}` replaces everything, so always send all the fields. It does not touch the question preview or the hints.
- **Question preview:** a separate large text area in the same editor, saved with its own button through `PUT .../preview` with `{ preview }`. Plain text up to 4000 characters; empty text removes it. This is what participants read before bidding, so label the field clearly ("Shown to participants") and show a character counter. The HackerRank link is never shown to bidders.
- **Hints:** three rows in the editor, one per corruption level (1, 2, 3), each with hint text (up to 2000 characters) and a password (up to 100 characters). Each row saves with `PUT .../hints/{level}`. Hints stay editable at any time, which is how typos get fixed during the event. Mark which levels are still empty, because a participant whose level has no hint cannot be helped (NOT_FOUND in the hint lookup).
- **Locking:** a problem that belongs to a round can only be created, edited, moved, deleted or have its question preview changed while that round is in PREVIEW. After that the server answers 409 INVALID_PHASE. Disable those controls when the phase is anything else, and explain why ("Locked: bidding has started"). Hints are the exception and stay editable.
- **Winners:** after settlement, a Winners tab on each problem lists the winners (name, HackerRank ID, price paid) from `GET .../winners`. It is empty before settlement.

5.3 Participant manager

- **Purpose:** find a participant quickly and act on them.
- **Shows:** a table of participants with name, HackerRank ID, email, coins and corruption level. Above it, a search box.
- **Calls:** `GET /api/admin/users` and `GET /api/admin/users?q=text`.
- **Search:** typing a name or HackerRank ID filters the table. Matching is by part of the text, in any letter case, with spaces around the text ignored. A blank search shows everyone; no match returns an empty list. Debounce the input

by about 300 milliseconds. Admins never appear in this list. Search is how an admin finds the right person without knowing any internal id: the row carries the id, and every action button below uses it.

- **Row actions:** Credit solved Bronze, Silver or Gold; Custom amount; Give hint; Devil's deal; Redemption; Set corruption level; Sign out everywhere. Details in 5.4.
- **Corruption level editor:** `PUT /api/admin/users/{userId}/corruption-level` with `{ level }` (1 to 3). Changes the level with no coin movement; use it to fix mistakes.
- **Sign out everywhere:** `POST /api/admin/users/{userId}/revoke-sessions` (204). Every earlier token for that person stops working and they must sign in again. Confirm first; use it for a cheater or a lost device.

5.4 Wallet and devil buttons

Every button below shows the confirm popup first and returns the new ledger entry (or, for the devil buttons, the entry plus the new corruption level).

Button	Call	Body	What happens	Disable or handle
Solved Bronze, Silver or Gold	`POST /api/admin/wallet/credit-preset`	`{ userId, tier }`	Credits the configured solve reward for that tier. One click per solved problem	None
Custom amount	`POST /api/admin/wallet/custom`	`{ userId, amount, reason }`	Positive credits, negative deducts. Reason required, up to 255 characters, and shown to the participant in their history, so write it for them	Zero gives INVALID_AMOUNT; a deduction below zero gives INSUFFICIENT_BALANCE
Give hint	`POST /api/admin/wallet/hint-deduction`	`{ userId, problemNumber }`	Deducts hintPrice. problemNumber is only a label for the log	INSUFFICIENT_BALANCE
Devil's deal	`POST /api/admin/wallet/devil-deal`	`{ userId }`	Corruption level +1, coins + devil.dealPayout	Disable at level 3 (CORRUPTION_LIMIT_REACHED)
Redemption	`POST /api/admin/wallet/redemption`	`{ userId }`	Corruption level -1, coins - devil.redemptionCost	Disable at level 1; INSUFFICIENT_BALANCE
Undo	`POST /api/admin/wallet/undo/{entryId}`	`{ reason }` (up to 200 characters)	Adds a REVERSAL entry. Undoing a devil entry also restores the previous corruption level	CONFLICT if the entry was already undone or is itself a reversal

Give hint flow. 1) The admin picks the participant and the problem (a dropdown from `GET /api/admin/problems`). 2) The panel calls `GET /api/admin/users/{userId}/hints/{problemId}`, which returns the hint text and password that match that participant's current corruption level, and shows them in a card so the admin can hand them over. A 404 NOT_FOUND means no hint is written for that level yet; show that and stop. 3) The admin confirms the deduction, and the panel calls `POST /api/admin/wallet/hint-deduction`. Fill `problemNumber` from the problem's position in the round (1 for the first), shown and editable.

After a round: paying solvers. The admin searches for each person who solved something, then presses the Solved Bronze, Silver or Gold button once per solved problem. No user id is ever typed. Use Custom amount for anything unusual.

5.5 Ledger viewer and undo

- **Purpose:** see every coin change and fix mistakes.
- **Shows:** a table, newest first: time, participant, type, amount, balance after, reason, who did it (`actorName`, empty for settlement) and whether it was undone. Filter by participant.
- **Calls:** `GET /api/admin/ledger?userId={id}&limit={n}` (limit defaults to 100, maximum 500).
- **Rules:** show an Undo button only when `type` is not REVERSAL and `undone` is false. Undo opens a popup asking for a reason, then calls the undo endpoint.

5.6 Audit log

- **Purpose:** a read-only record of who did what.
- **Shows:** a table, newest first: time, admin name and email, method, path and the HTTP status the call ended with (2xx went through, 4xx or 5xx was refused or failed).
- **Calls:** `GET /api/admin/audit?actorId={id}&limit={n}` (limit defaults to 100, maximum 500).
- **Rules:** every state-changing admin call is logged automatically, including refused attempts by participants. Request bodies are never stored, so use the ledger for coin details. There is nothing to create or edit here.

6. Phase by phase behaviour

The round phase decides what every screen shows and which buttons work. The phase moves only forward, only when the admin presses a button, and the participant app finds out by polling `GET /api/rounds/current` every 5 seconds. The order is fixed: PREVIEW, BIDDING, SETTLEMENT, SOLVING, RESULTS, and then the admin starts the next round, which begins again at PREVIEW.

What each side sees and can do.

Phase	Participant sees	Participant can	Admin can
No round yet	"The event has not started yet" (the API answers ROUND_NOT_FOUND, which is normal)	Read the Rules page, edit the profile	Create and prepare problems in the bank, then start round 1
PREVIEW	Problem cards with title, category, tier, teaser, question preview, winners count and reserve price	Study the problems. No bid input; show "Bidding opens soon"	Add, edit, move and delete problems and previews; write hints; set a bidding deadline; open bidding (needs at least one problem)
BIDDING	The sealed bid screen with a countdown to the deadline when one is set	Place and change bids	Set or change the deadline, pause, close bidding. Problems are now locked (hints are not)
SETTLEMENT	"Bidding closed, results coming"	Nothing to do but wait	Run settlement. No advance button in this phase
SOLVING	Their own results: won problems with the HackerRank link and price paid, and lost or unaffordable bids	Open won problems on HackerRank. Admins then credit solved problems	Show results (advance), read the settlement summary and winners, credit coins, give hints
RESULTS	The same as solving, plus the leaderboard	Read results and the leaderboard	Start the next round

Paused rounds. While `paused` is true, phase buttons are disabled for the admin and bids are refused for participants. Show a clear banner on both sides ("Round paused. Bidding is on hold") and keep polling, because the admin can resume at any time. Pause is idempotent: pausing a paused round, or resuming a running one, is not an error.

Bidding deadline. When `biddingDeadline` is set, the bid screen shows a countdown computed with the server clock offset (section 3). At zero, lock the inputs at once without waiting for the next poll. The deadline does not change the phase; the round stays in BIDDING until the admin closes it, but the server refuses bids at or after the deadline.

Banner texts. Use one phase banner component driven by `phase` and `paused`, so the wording is identical everywhere:

- PREVIEW: "Round N is in preview. Study the problems; bidding opens soon."
- BIDDING: "Bidding is open. Place your sealed bids."
- SETTLEMENT: "Bidding is closed. Results are coming."
- SOLVING: "Results are in. Solve your problems on HackerRank."
- RESULTS: "Round N is over. Check the leaderboard."

What each endpoint returns in each phase.

Endpoint	Before bidding	Bidding	After settlement
`GET /api/rounds/current/problems`	Problems (empty list if none)	Problems	Problems
`GET /api/me/bids`	Empty list	The participant's pending bids	Bids with WON, LOST or UNAFFORDABLE; won bids carry price paid and link
`GET /api/admin/problems/{id}/winners`	Empty list	Empty list	The winners
`GET /api/admin/rounds/current/settlement`	INVALID_PHASE	INVALID_PHASE	The summary
`PUT .../bid`	BIDDING_CLOSED	Accepted unless paused or past the deadline	BIDDING_CLOSED

7. Pop-ups and notifications

Every coin or level change, every settlement result and every phase change reaches the participant as an event. The frontend turns personal events into transaction pop-ups and keeps all events in the notification bell. The server writes the events; the frontend only polls and displays.

7.1 The event feed

Endpoint: `GET /api/me/events?after={id}&limit={n}`. It returns an array of EventResponse, oldest first. `limit` defaults to 50. Event ids only ever increase, so the id of the last event you handled is your cursor.

- **First call after sign-in** (no `after`): returns the 20 most recent events. Use them to fill the bell list and to set `lastId`. **Do not show pop-ups for these**; they are history.
- **Every later call** uses `after=lastId` and returns only newer events. Handle them in order and update `lastId` after each one.
- Optional but recommended: keep `lastId` in `sessionStorage` (it is not sensitive). After a page reload, resume from it so an event that arrived during the reload still pops up instead of silently landing in the bell.
- Poll every 5 seconds while the app is open and the tab is visible. When the tab becomes visible again, poll once immediately.

7.2 What to do with each event

Field	Meaning	UI use
`id`	Increasing number	Cursor for `after`
`type`	One of the 11 event types below	Pick the pop-up style and icon
`broadcast`	true means sent to everyone; false means personal	false: pop-up and bell. true: bell only
`message`	Ready-to-display sentence written by the server	Show it as is. Never rebuild the sentence from the other fields. Render as plain text
`amount`	Signed coin change, or null	Show big and coloured: green for positive, red for negative
`balanceAfter`	Wallet balance after the change, or null	Show "Balance: N coins" under the amount
`corruptionLevel`	New level for DEVIL_DEAL and REDEMPTION (also set when an undo restored a level), else null	Update the level badge at once, without waiting for the next `GET /api/me`
`problemId`	Set on BID_WON, BID_LOST, BID_UNAFFORDABLE	Use it to link the pop-up to the problem in My results
`createdAt`	ISO timestamp	Show in the bell list in local time

After any **personal** event arrives, refetch `GET /api/me` so coins and level are exact.

7.3 The eleven event types

Type	Broadcast	When it is created	Pop-up style
BID_WON	No	Settlement: the participant won a problem. amount is minus the price paid	Positive (celebration). Link: "Open my problems"
BID_LOST	No	Settlement: the bid did not win	Neutral
BID_UNAFFORDABLE	No	Settlement: would have won but the balance no longer covered the price. Nothing charged	Warning. Explain that no coins were taken
PRESET_CREDIT	No	Admin pressed Solved Bronze, Silver or Gold	Positive
HINT_DEDUCTION	No	Admin deducted the hint price	Negative or neutral
DEVIL_DEAL	No	Devil’s deal: coins up, level up. Carries the new level	Positive, devil theme. Show the new level
REDEMPTION	No	Redemption: coins down, level down. Carries the new level	Negative or neutral, "soul bought back" theme
CUSTOM_ADJUSTMENT	No	Admin custom amount. The message includes the admin’s reason	Colour by the sign of amount
REVERSAL	No	Admin undid an earlier change. Carries the restored level when a devil entry was undone	Colour by the sign of amount
ROUND_STARTED	Yes	Admin started a round	Bell only
PHASE_CHANGED	Yes	Admin advanced the phase or ran settlement	Bell only. Also triggers a round refetch

7.4 Pop-up behaviour

- Mount one pop-up layer at the top of the app so it works on every screen, including the bid screen. Use `role="status"` and `aria-live="polite"` so screen readers announce it.
- **Queue them.** Settlement can create several events in the same poll (one per bid the participant placed). Show one pop-up at a time with a counter such as "1 of 3". The participant dismisses with a tap, or it closes by itself after about 6 seconds if more are waiting. Never stack pop-ups on top of each other and never block the bid input for longer than a tap.
- Show: an icon for the type, the `message`, the signed amount in large coloured digits, and the balance after. For DEVIL_DEAL and REDEMPTION also show the corruption level change ("Level 1 to 2").
- BID_WON, BID_LOST and BID_UNAFFORDABLE pop-ups get a button to open My results.
- Pop-ups appear for events that happen while the app is open. The bell is the safety net for everything the participant missed.

7.5 The notification bell

- Shows an unread count and, on tap, a list of all events (personal and broadcast), newest first, with time and message. There is no "mark as read" endpoint: track read state on the client (for example the highest event id seen when the panel was last opened).
- Broadcast events ROUND_STARTED and PHASE_CHANGED appear only here. They also tell you to refetch `GET /api/rounds/current` immediately instead of waiting for the next 5-second tick.
- Keep the last 50 events in memory; older ones can be dropped.

8. A full event day, in order

This walkthrough shows how the screens and calls fit together on a real day. Use it as an acceptance script: if each step works, the event works.

8.1 Before the day

Step	Who	What happens	Screen and calls
1	Organisers	Decide coin prices, rewards, hint price and devil rates and set them in the backend config. Set the environment variables (Google client ID, JWT secret, admin emails, fallback passcode)	Backend only. The frontend reads the result from `GET /api/config`
2	Admin	Create the problems of round 1 in the Problem bank, with title, tier, slots, HackerRank link, teaser and question preview	`POST /api/admin/problems`, `PUT .../preview`
3	Admin	Write the three hint versions and passwords for every problem	`PUT .../hints/{level}` (levels 1, 2, 3)
4	Everyone	Dry-run with test accounts: a real Google sign-in, the profile form, a full round. This is the first real use of Google sign-in	All screens. See the test plan in section 12.4

8.2 Opening the event

Step	Who	What happens	Screen and calls
5	Participants	Open the app, sign in with Google, fill the profile form (name, HackerRank ID)	Login, Profile form: `POST /api/auth/google`, `PUT /api/me/profile`
6	Participants	See "The event has not started yet" until round 1 exists	Dashboard (ROUND_NOT_FOUND state)
7	Admin	Give everyone their starting coins, one participant at a time, with Custom amount (see the open decision in section 12.2)	Participant manager: search, then `POST /api/admin/wallet/custom`
8	Admin	Start round 1. The round begins in PREVIEW	Round controls: `POST /api/admin/rounds`
9	Participants	Bell shows "Round 1 has started". The dashboard shows PREVIEW. They read the problem cards and question previews	Dashboard, Round preview

8.3 One round

Step	Who	What happens	Screen and calls
10	Admin	Optionally set a bidding deadline, then press Open bidding. The server refuses if the round has no problems	`PUT .../bidding-deadline`, `POST .../advance`
11	Participants	Bid screen opens with the countdown. They enter and save bids; the summary bar shows total and coins left	`PUT .../problems/{id}/bid`
12	Admin	If something goes wrong, Pause. Participants see the paused banner and cannot bid; Resume continues	`POST .../pause`, `POST .../resume`
13	Admin	At the deadline, press Close bidding. The round moves to SETTLEMENT	`POST .../advance`
14	Admin	Press Run settlement. Winners are charged, results are written, the round moves to SOLVING. The summary table appears	`POST .../settle`
15	Participants	Pop-ups arrive: won, lost or unaffordable. My results lists the bids; winners get the price paid and the HackerRank link	Pop-ups, My results: `GET /api/me/bids`
16	Participants	Solve the problems on HackerRank (outside the app)	-

Step	Who	What happens	Screen and calls
17	Admin	During solving: give hints on request. Look up the right hint for the participant's level, hand it over, then confirm the deduction	Participant manager: `GET .../hints/{problemId}`, `POST .../hint-deduction`
18	Admin	During solving: run devil's deals and redemptions for participants who ask. Fix mistakes with Undo	`POST .../devil-deal`, `POST .../redemption`, `POST .../undo/{entryId}`
19	Admin	When people finish, press Solved Bronze, Silver or Gold on each solver, once per solved problem	`POST .../credit-preset`
20	Admin	Press Show results. The round moves to RESULTS	`POST .../advance`
21	Participants	Check the leaderboard and their history	Leaderboard, History
22	Admin	Press Start next round. Round 2 starts in PREVIEW and the loop repeats from step 10 (add round 2 problems first)	`POST /api/admin/rounds`

8.4 When things go wrong on the day

- **Google sign-in is down:** admins use the fallback sign-in (allowlisted email plus the shared passcode). Participants cannot sign in without Google.
- **A participant is misbehaving or lost a device:** Participant manager, "Sign out everywhere". They sign in again.
- **A coin action went to the wrong person:** Ledger viewer, Undo with a reason. The original entry stays and a reversal is added.
- **A hint or question preview has a typo:** hints can be edited any time. The question preview is locked once bidding opens; tell participants, or correct it in the next round.
- **Two admins clicked at once:** one gets CONCURRENT_UPDATE; refetch and retry.

9. Edge cases and what the UI does

Each row is a situation that will happen during a real event. The backend already handles all of them safely; the UI's job is to say the right thing and not make it worse.

Situation	What the backend does	What the UI does
Before round 1	GET /api/rounds/current answers 404 ROUND_NOT_FOUND	Friendly "event has not started" screen, no error styling
Deadline passes while a participant is typing	Rejects the bid with BIDDING_CLOSED	Lock all inputs, keep the typed text visible, refetch the round, show "Bidding closed"
Admin pauses during bidding	Rejects bids with BIDDING_CLOSED while paused	Read-only inputs and a paused banner. Keep polling so it unlocks on resume
Participant's bids add up to more than their coins	Rejects with BID_EXCEEDS_BALANCE	Show the live total and block Save before the call; also handle the error
Bid below the reserve price	VALIDATION_FAILED with details.amount	Show the message under the input; check client-side first
Admin deducts coins after a participant bid	At settlement the winner who cannot pay becomes UNAFFORDABLE; nothing charged; slot not reassigned	Explain it on the result and in the pop-up
Tie on the winning bids	Earlier submission wins. Re-saving the same amount does not change submission time; changing the amount does	Rules page explains it. Do not disable edits, but warn that changing the amount resets the queue position
Fewer bids than slots plus one	Winners pay the reserve price	Result text shows the price actually paid, not the bid
Token about to expire	Refresh gives a new token	Refresh silently under 15 minutes left. Never interrupt the participant
Token expired or revoked	401 TOKEN_EXPIRED or TOKEN_INVALID	Clear the token, go to login, keep the participant's unsaved bid text if easy

Situation	What the backend does	What the UI does
Participant has not finished the profile	403 PROFILE_INCOMPLETE on everything else	Redirect to the profile form from any screen
HackerRank ID already taken	CONFLICT with details.hackerRankId	Message next to the field; let them fix a typo or contact an organiser
Backend or database down	503 SERVICE_UNAVAILABLE (or network error)	Show "Try again shortly". Never sign the user out. Keep polling with a back-off
Participant’s clock is wrong	Server judges deadlines by its own clock	Use the serverTime offset for every countdown
Two admins click the same phase button	One succeeds, the other gets CONCURRENT_UPDATE	Refetch the round and show the new state
Settle pressed twice	Second call answers ALREADY_SETTLED or CONCURRENT_UPDATE; coins charged once	Disable the button while the call runs; on error refetch and show the summary
Open bidding with no problems	409 INVALID_PHASE	Show the message; link to the Problem bank
Editing a problem after bidding opened	409 INVALID_PHASE	Disable the controls in advance with an explanation. Hints stay editable
Devil’s deal at level 3, redemption at level 1	409 CORRUPTION_LIMIT_REACHED	Disable those buttons using the participant’s level
Redemption or hint with too few coins	409 INSUFFICIENT_BALANCE	Show the message; the confirm popup should show the current balance
Undo the same entry twice, or undo a reversal	409 CONFLICT	Hide Undo when undone is true or the type is REVERSAL; handle the error anyway
Undo of a devil entry that cannot restore the level	409 CORRUPTION_LIMIT_REACHED and the whole undo is rolled back, coins included	Show the message. Fix the level first with the corruption level editor, then undo
Hint lookup for a level with no hint written	404 NOT_FOUND	Say which level is missing, link to the problem editor
Double click on Confirm	Two calls would create two ledger entries	Disable Confirm while the request runs. Treat it as a hard rule for every wallet button
An admin account opens the bid screen or calls the bid endpoint	403 FORBIDDEN	Never show the participant app to admins
Tab left open for hours	Data may be stale	Refetch round, me and events when the tab becomes visible
Settlement creates many events at once	One event per bid, delivered in one poll	Queue pop-ups with a counter (section 7.4)
Leaderboard ties	Equal coins share a rank	Show the same rank for ties and expect gaps (1, 1, 3)

10. Full API reference

Every endpoint, grouped by who may call it. Bodies and responses use the types in section 11. All paths start with `/api` and use the Bearer header unless marked public. Anything under `/api/admin` needs the ADMIN role.

10.1 Public

Endpoint	Body	Success	Notes
`GET /api/health`	-	`{ status: "UP", time }`	No auth
`POST /api/auth/google`	`GoogleLoginRequest`	`AuthResponse`	Errors: INVALID_GOOGLE_TOKEN, EMAIL_NOT_VERIFIED, VALIDATION_FAILED
`POST /api/auth/admin-fallback`	`AdminFallbackRequest`	`AuthResponse` (role ADMIN)	INVALID_CREDENTIALS; TOO_MANY_REQUESTS after 5 failures per email in 5 minutes

10.2 Any signed-in user

Endpoint	Body	Success	Used by
`POST /api/auth/refresh`	-	`AuthResponse`	Session refresh
`GET /api/config`	-	`GameConfigResponse`	Rules page, confirm popups. Fetch once
`GET /api/me`	-	`MeResponse`	Top bar, dashboard
`PUT /api/me/profile`	`ProfileRequest`	`MeResponse`	Profile form. CONFLICT when the HackerRank ID is taken
`GET /api/me/history`	-	`LedgerEntryResponse[]`	History, newest first
`GET /api/me/events?after&limit`	-	`EventResponse[]`	Pop-ups and bell, oldest first, limit defaults to 50
`GET /api/rounds/current`	-	`RoundResponse`	Phase banner. 404 ROUND_NOT_FOUND before round 1
`GET /api/rounds/current/problems`	-	`ProblemPreviewResponse[]`	Problem cards, ordered by id
`PUT /api/rounds/current/problems/{id}/bid`	`BidRequest`	`BidResponse`	Participants only. Place or replace a bid
`GET /api/me/bids`	-	`BidResponse[]`	Bid screen, My results, My problems
`GET /api/leaderboard`	-	`LeaderboardEntry[]`	Leaderboard

10.3 Admin: rounds and auction

Endpoint	Body	Success	Rules
`POST /api/admin/rounds`	-	`201 RoundResponse`	Starts round 1 or the next round. INVALID_PHASE unless there is no round or the latest is in RESULTS
`POST /api/admin/rounds/current/advance`	-	`RoundResponse`	Next phase. INVALID_PHASE if paused, finished, leaving PREVIEW with no problems, or in SETTLEMENT (use settle)
`POST /api/admin/rounds/current/pause`	-	`RoundResponse`	Idempotent

Endpoint	Body	Success	Rules
`POST /api/admin/rounds/current/resume`	-	`RoundResponse`	Idempotent
`PUT /api/admin/rounds/current/bidding-deadline`	`BiddingDeadlineRequest`	`RoundResponse`	Only in PREVIEW or BIDDING. Must be in the future. null clears it
`POST /api/admin/rounds/current/settle`	-	`SettlementSummaryResponse`	Only in SETTLEMENT and not paused. ALREADY_SETTLED if repeated. Everything happens in one transaction
`GET /api/admin/rounds/current/settlement`	-	`SettlementSummaryResponse`	INVALID_PHASE before settlement

10.4 Admin: problem bank

Endpoint	Body	Success
`GET /api/admin/problems` (`?roundId` optional)	-	`AdminProblemResponse[]`
`GET /api/admin/problems/{id}`	-	`AdminProblemResponse`
`POST /api/admin/problems`	`ProblemRequest`	`201 AdminProblemResponse`
`PUT /api/admin/problems/{id}`	`ProblemRequest` (full replace, leaves the preview alone)	`AdminProblemResponse`
`DELETE /api/admin/problems/{id}`	-	`204`
`PUT /api/admin/problems/{id}/preview`	`ProblemPreviewRequest`	`AdminProblemResponse`
`PUT /api/admin/problems/{id}/hints/{level}`	`HintRequest` (level 1 to 3)	`HintResponse`
`GET /api/admin/problems/{id}/winners`	-	`WinnerResponse[]` (empty before settlement)

Create, edit, move, delete and preview changes on a problem that belongs to a round are allowed only while that round is in PREVIEW (otherwise 409 INVALID_PHASE). Hints are editable at any time.

10.5 Admin: participants

Endpoint	Body	Success	Notes
`GET /api/admin/users` (`?q=text`)	-	`MeResponse[]`	Participants only, sorted by name. q matches part of the name or HackerRank ID, any case; blank q returns everyone
`GET /api/admin/users/{userId}/hints/{problemId}`	-	`HintResponse`	The version and password for the participant's current level. 404 NOT_FOUND if that level has no hint
`PUT /api/admin/users/{userId}/corruption-level`	`CorruptionLevelRequest`	`MeResponse`	No coin movement
`POST /api/admin/users/{userId}/revoke-sessions`	-	`204`	Every earlier token is rejected with TOKEN_INVALID. 404 USER_NOT_FOUND for an unknown id

10.6 Admin: wallet, ledger and audit

Endpoint	Body	Success	Effect
`POST /api/admin/wallet/credit-preset`	`PresetCreditRequest`	`LedgerEntryResponse`	Credits config.solveReward for the tier
`POST /api/admin/wallet/hint-deduction`	`HintDeductionRequest`	`LedgerEntryResponse`	Deducts config.hintPrice

Endpoint	Body	Success	Effect
`POST /api/admin/wallet/custom`	`CustomAmountRequest`	`LedgerEntryResponse`	Positive credits, negative deducts, zero is INVALID_AMOUNT
`POST /api/admin/wallet/devil-deal`	`DevilActionRequest`	`DevilActionResponse`	Level +1, coins + dealPayout. CORRUPTION_LIMIT_REACHED at level 3
`POST /api/admin/wallet/redemption`	`DevilActionRequest`	`DevilActionResponse`	Level -1, coins - redemptionCost. CORRUPTION_LIMIT_REACHED at level 1
`POST /api/admin/wallet/undo/{entryId}`	`UndoRequest`	`LedgerEntryResponse`	Adds a REVERSAL. CONFLICT if already undone or itself a reversal
`GET /api/admin/ledger?userId&limit`	-	`AdminLedgerEntryResponse[]`	Newest first. limit defaults to 100, maximum 500
`GET /api/admin/audit?actorId&limit`	-	`AuditLogResponse[]`	Newest first. limit defaults to 100, maximum 500

The admin id used for wallet calls is taken from the signed-in admin on the server side; the frontend never sends it. A wallet call that would take a balance below zero answers INSUFFICIENT_BALANCE and changes nothing.

11. Data types

11.1 Enums

```
type Role       = 'PARTICIPANT' | 'ADMIN';
type Tier       = 'BRONZE' | 'SILVER' | 'GOLD';
type RoundPhase = 'PREVIEW' | 'BIDDING' | 'SETTLEMENT' | 'SOLVING' | 'RESULTS';  // forward only
type BidStatus  = 'PENDING' | 'WON' | 'LOST' | 'UNAFFORDABLE';
type LedgerType = 'PRESET_CREDIT' | 'HINT_DEDUCTION' | 'BID_SETTLEMENT' | 'DEVIL_DEAL'
                | 'REDEMPTION' | 'CUSTOM' | 'REVERSAL';
type EventType  = 'BID_WON' | 'BID_LOST' | 'BID_UNAFFORDABLE' | 'PRESET_CREDIT'
                | 'HINT_DEDUCTION' | 'DEVIL_DEAL' | 'REDEMPTION' | 'CUSTOM_ADJUSTMENT'
                | 'REVERSAL' | 'ROUND_STARTED' | 'PHASE_CHANGED';
```

Corruption level is an integer from 1 to 3 and everyone starts at 1. UNAFFORDABLE means the participant would have won but their balance no longer covered the price at settlement.

11.2 Errors, auth, config and profile

```
interface ErrorResponse {
  timestamp: string; status: number; code: ErrorCode;
  message: string;                    // safe to show to the user
  path: string;
  details?: Record<string, string>;   // field name -> problem; omitted when empty
}
interface GoogleLoginRequest { idToken: string }
interface AdminFallbackRequest { email: string; passcode: string }
interface AuthResponse { token: string; expiresAt: string; profileComplete: boolean; role: Role }

interface TierAmounts { bronze: number; silver: number; gold: number }
interface GameConfigResponse {
  basePrice: TierAmounts;           // default reserve price per tier
  solveReward: TierAmounts;         // credit for "solved Bronze/Silver/Gold"
  hintPrice: number;                // same for every problem
  devil: { dealPayout: number; redemptionCost: number };  // redemptionCost > dealPayout
  minCorruptionLevel: number;       // 1
  maxCorruptionLevel: number;       // 3
}
interface MeResponse {
  id: number; email: string;
  name: string | null;              // null until the profile form is saved
  hackerRankId: string | null;
  role: Role;
  corruptionLevel: number;          // 1..3
  coins: number;
  profileComplete: boolean;
}
interface ProfileRequest { name: string; hackerRankId: string }  // max 100 / max 50, no spaces
```

11.3 Ledger and events

```
interface LedgerEntryResponse {
  id: number; type: LedgerType;
  amount: number;                   // signed: positive credit, negative deduction
  balanceAfter: number; reason: string;
  reversalOf: number | null; createdAt: string;
}
interface AdminLedgerEntryResponse extends LedgerEntryResponse {
  userId: number; userName: string | null;
  actorName: string | null;         // admin who did it; null for system actions (settlement)
  undone: boolean;                  // true if a reversal already exists for this entry
}
interface EventResponse {
  id: number;                       // increasing; use as the `after` cursor
  type: EventType;
  broadcast: boolean;               // true = everyone (bell only); false = personal (pop-up)
  message: string;                  // ready to display
  amount: number | null; balanceAfter: number | null;
  corruptionLevel: number | null;   // new level, for DEVIL_DEAL / REDEMPTION
  problemId: number | null;         // for BID_* events
  createdAt: string;
}
```

11.4 Rounds, problems and bids

```
interface RoundResponse {
  id: number; number: number; phase: RoundPhase; paused: boolean;
  phaseChangedAt: string;
  biddingDeadline: string | null;   // null = no deadline set
  serverTime: string;               // for clock-skew correction
}
interface ProblemPreviewResponse {  // participant view: no link, hints or passwords
  id: number; title: string; category: string; tier: Tier;
  teaser: string | null;
  preview: string | null;           // question preview text written by the admin
  slots: number;                    // number of winners
  basePrice: number;                // reserve price
}
interface HintResponse { corruptionLevel: number; hintText: string; password: string }  // admin only
interface AdminProblemResponse {
  id: number; title: string; category: string; tier: Tier; slots: number;
  hackerRankLink: string; basePrice: number;
  teaser: string | null; preview: string | null;
  roundId: number | null;           // null = bank only
  hints: HintResponse[];            // up to 3, ordered by corruptionLevel
}
interface ProblemRequest {
  title: string;                    // required, max 150
  category: string;                 // required, max 60
  tier: Tier; slots: number;        // slots 1..100
  hackerRankLink: string;           // required, max 500, ^https?://\S+$
  basePrice?: number | null;        // >= 0; omitted = tier default from /api/config
  teaser?: string | null;           // max 500
  roundId?: number | null;          // omitted = bank only
}
interface ProblemPreviewRequest { preview: string | null }   // max 4000; null or blank removes
interface HintRequest { hintText: string; password: string } // max 2000 / max 100
interface WinnerResponse { userId: number; name: string | null;
                           hackerRankId: string | null; pricePaid: number }
interface BidRequest { amount: number }                      // whole coins, >= 1
interface BidResponse {
  problemId: number; problemTitle: string; amount: number; status: BidStatus;
  pricePaid: number | null;         // what a winner paid (clearing price, not their bid)
  hackerRankLink: string | null;    // only when status is WON
  submittedAt: string;
}
interface BiddingDeadlineRequest { deadline: string | null } // ISO instant, or null to clear
interface SettlementSummaryResponse {
  roundId: number; roundNumber: number; totalCollected: number;
  problems: { problemId: number; title: string; bidCount: number; winnerCount: number;
              unaffordableCount: number; clearingPrice: number }[];
}
interface LeaderboardEntry { rank: number; userId: number; name: string | null;
                             hackerRankId: string | null; coins: number }
```

11.5 Admin wallet and audit

```
interface PresetCreditRequest  { userId: number; tier: Tier }
interface HintDeductionRequest { userId: number; problemNumber: number }   // >= 1, label only
interface CustomAmountRequest  { userId: number; amount: number; reason: string } // reason max 255
interface UndoRequest          { reason: string }                          // max 200
interface DevilActionRequest   { userId: number }
interface DevilActionResponse  { entry: LedgerEntryResponse; corruptionLevel: number }
interface CorruptionLevelRequest { level: number }                         // 1..3
interface AuditLogResponse {
  id: number; actorId: number; actorName: string | null; actorEmail: string;
  method: 'POST' | 'PUT' | 'PATCH' | 'DELETE';
  path: string;                    // e.g. "/api/admin/rounds/current/settle"
  status: number;                  // 2xx went through, 4xx/5xx was refused or failed
  createdAt: string;
}
```

12. Design guidance, decisions, build order and tests

12.1 Design guidance

- **Participant app: mobile first.** One column, large tap targets, the coin balance and phase always visible, numeric keyboard on the bid input. Phones will be on event Wi-Fi, so keep screens light and polling cheap.
- **Admin panel: desktop first.** Dense tables, keyboard-friendly forms, the round state always visible in the header. Optimise for speed under pressure: a search box that is focused on load, one-click row actions, and no typing of ids.
- **Colour has meaning.** Green for credits, red for deductions, amber for warnings (paused, UNAFFORDABLE), neutral for information. Never rely on colour alone: add a sign (+ or -) and an icon.
- **Tier badges.** Bronze, Silver and Gold as three consistent badge colours used on cards, results and admin tables.
- **Numbers.** Show coins as whole numbers with a thousands separator. Put the unit near the first occurrence on each screen.
- **Every screen has four states:** loading, empty, error and normal. The empty and "not started" states are friendly, not red.
- **Destructive or coin-moving actions always confirm,** name the person and the exact amount, and disable the confirm button while the call is running.
- **Devil theme.** The event is called Deal with Devil; a dark palette with one strong accent colour and a light touch of theming on the devil pop-ups fits. Do not let the theme hurt readability of numbers.
- **Accessibility.** Real buttons and labels, visible focus, sufficient contrast, `aria-live` on pop-ups and the countdown, and no information carried by colour only.
- **Time.** Show timestamps in local time. Countdowns use the server offset (section 3).

12.2 Open decisions

These are not frontend choices, but they change what the screens show. Build so that none of them is hard-coded.

Decision	Status	Effect on the frontend
Coin prices, tier rewards, hint price, preset amounts	Open. Placeholders in the backend config: base price 50/100/200, solve reward 100/200/400, hint 30	None if you read everything from GET /api/config
Devil payout and redemption cost	Open. Placeholders: deal pays 100, redemption costs 150 (redemption must cost more)	Same: read from config; never print fixed numbers
How participants get their first coins	Open. There is no starting grant and no bulk-credit endpoint; today an admin uses Custom amount per person	If the organisers want a bulk grant, ask for a backend endpoint; otherwise plan time on the day for step 7 of section 8
Reserve price and tie rules	Implemented as: reserve is the problem's base price, bids below it are rejected, earlier bid wins ties. To be confirmed with organisers	The Rules page text must match the final decision
Slots per problem, rounds, problems per round	Open	None; all data driven

Decision	Status	Effect on the frontend
How the devil appears (every round, timed, on request)	Open	Today only the admin triggers it. If participants should request it in the app, that is a new feature
Admins and HackerRank ID	Open. The profile gate applies to participants only	None for now
Mystery assets	On hold	Not in scope
Google OAuth in production	Open. Consent screen must be In production; the real React origin must be added as an authorised origin and as an allowed CORS origin	Needed before the real event; test login on the real domain early
Manual override of a settlement result	Not built. Only undo of coin entries and the level editor exist	No override button in the admin panel

12.3 Suggested build order

Step	Build	Done when
1	Project setup, API client, error handler, session (token in memory or sessionStorage), role router	A fixture-backed login reaches the right empty app
2	Login with Google, fallback admin sign-in, profile form, config and me loading, refresh timer	A real participant and a real admin can sign in against the backend
3	Participant shell: top bar, phase banner, dashboard, server-clock countdown	Phase changes made by an admin show up within 5 seconds
4	Admin round controls and settlement summary	An admin can start a round, open and close bidding, pause and settle
5	Admin problem bank with previews and hints, locking behaviour	A full set of problems can be created and edited in PREVIEW; controls lock afterwards
6	Participant problem cards and sealed bid screen	Bids save, totals update live, every bid error is handled
7	My results, My problems, history, leaderboard	After settlement the winner sees price paid and the link
8	Event feed: pop-up queue, bell	Settlement shows one pop-up per bid; reload does not replay old events
9	Admin participant manager, wallet and devil buttons with confirm popups, hint flow, level editor, sign out everywhere	A whole round of payouts can be done without typing an id
10	Ledger viewer with undo, audit log	A wrong credit can be found and undone
11	Rules page driven by config, polish, empty and error states, accessibility pass	Every screen has all four states
12	Dry-run with test accounts (section 12.4)	The acceptance script in section 8 passes end to end

> **Tip.** Steps 3 to 8 are in the order the event happens, which means each step can be tested by hand against the real backend as soon as it is built. Use fixtures only for what is not built yet.

12.4 Mock data and test plan

Fixtures. Use MSW or JSON fixtures typed with section 11. Minimum set to cover every screen: one participant and one admin MeResponse; a RoundResponse in each of the five phases; three ProblemPreviewResponse (one per tier); a BidResponse in each status; a LedgerEntryResponse of each type; an EventResponse of each type (personal and broadcast); one SettlementSummaryResponse; a LeaderboardEntry list with a tie; and an ErrorResponse for every code in section 3.

Manual test plan against the real backend. Do this at least once before the event, with two or three test Google accounts and one admin account:

- Sign in with Google as a participant and as an admin; confirm each lands in the right app. Test the fallback admin sign-in.
- Complete the profile form; try a duplicate HackerRank ID and see the field error.
- As admin: create problems, write previews and hints, give the test participants coins with Custom amount, start round 1.
- As participants: place bids that total more than the balance, a bid below the reserve, and a valid bid. Change a bid. Check the totals bar.
- Set a deadline 2 minutes ahead, watch the countdown reach zero on two devices, and confirm both lock. Pause and resume during bidding.
- Close bidding and run settlement. Check winners, price paid, the pop-up queue, and the admin summary. Press settle again to confirm coins are charged once.
- Make one winner unaffordable (deduct coins after they bid) and check the result and message.
- Run a devil's deal, a redemption, a hint (with hint lookup) and a preset credit; undo one of them and confirm the level is restored for devil entries.
- Sign a participant out everywhere and confirm they are sent to login on their next request.
- Leave a participant tab open past the token lifetime and confirm the silent refresh keeps them signed in.
- Stop the backend for a minute and confirm the UI shows "Try again shortly" and recovers without signing anyone out.

12.5 Definition of done and handover

- Every screen in sections 4 and 5 exists and has loading, empty, error and normal states.
- Every error code in section 3 is handled in one place and shows the server message.
- No price, reward, devil rate or limit is hard-coded; all come from `GET /api/config`.
- Every wallet button confirms first and cannot be double-sent.
- The acceptance script in section 8 passes against the real backend with real Google sign-in.
- The production React origin is set on the backend (`CORS_ALLOWED_ORIGINS`) and in the Google Cloud authorised origins.
- The token is never in `localStorage` and no third-party script is on the page.

Where to look for more. `docs/frontend-api-contract.md` is the endpoint reference this document is built on, `docs/hacker-auction-context_2.md` explains the design decisions behind the event, and `docs/roadmap.md` tracks what is built and what is still open on the backend.

Deal with Devil: Frontend Build Spec

Phase 9 update: exactly 3 problems per person

Oct 21, 2026 | @Vishesh | Read this together with the spec above. Where this update and an earlier section disagree, this update wins.

What changed. The backend now guarantees that every participant ends each round with exactly 3 problems. The number comes from `game.wins-per-round` (3 today), so never hardcode it: read it from `winsPerRound` in the admin responses and treat "3" in this text as that value. Settlement no longer runs one auction per problem; it runs one allocation for the whole round with a cap of 3 wins per person, and an admin step after settlement tops up anyone who is short. All request and response shapes are also in `docs/frontend-api-contract.md`, which stays the endpoint reference.

1. The new rules in plain words

- **Slots are automatic.** The admin presses one button; total slots become 3 x participants, spread evenly over the problems. Nobody edits slots by hand.
- **Nobody wins more than 3 problems in a round.** Bids are ranked by amount, then earlier submission. A bid wins only if the person has fewer than 3 wins and the problem still has a free slot.
- **Price per problem** = the first bid turned away because the problem was full, otherwise the reserve price. Every winner of that problem pays it. A bid that loses only because its owner already has 3 wins never sets anyone's price.
- **Anyone short of 3** (for example someone who never bid) is topped up by the admin's Auto fill at the reserve price. Whoever Auto fill cannot finish is finished by hand (Manual assignment).
- **Bid limit changed.** Only your 3 highest bids have to fit in your coins, because you can never be charged for more than 3 problems.

2. What changes in the existing sections

Section	Was	Now
2 Core concepts, Rules page (4.9)	Top "slots" bidders win and pay the (slots+1)th bid. "All your bids together cannot exceed your coins."	Same price idea, but you can win at most 3 problems per round and you always end with exactly 3. Rule text: "Your 3 highest bids together cannot exceed your coins." Add: "If you win fewer than 3, the organisers give you more at the reserve price."
4.4 Problem cards	Shows slots as "N winners".	Unchanged. Slots now come from the Auto slots button, so the numbers can differ between problems and rounds.
4.5 Sealed bid screen	Running total of all bids must stay within coins.	Running total = sum of the 3 highest saved bids (including the one being typed). Disable Save only when that total exceeds coins. Show a banner "You have bid on X of Y problems" and encourage bidding on every problem (the allocation works best when everyone does). BID_EXCEEDS_BALANCE: show the server message.
4.6 My results / my problems	Won problems vary per person.	After the admin finishes the round every participant has exactly 3 WON bids. Problems given by Auto fill or by an admin look like normal wins (status WON, pricePaid = reserve price, HackerRank link) and arrive as a BID_WON pop-up with the text "You were given ...". If the person has fewer than 3 in SOLVING, show "The organisers are still assigning your problems."
5.1 Round controls	Open bidding refused only when the round has no problems.	Also refused with INVALID_PHASE when the round has fewer than 3 problems or fewer slots than 3 x participants. Show the message and point to the Slots card (press Auto slots again, then open bidding).
5.2 Problem bank, slots field	Required 1 to 100, hand-edited.	Keep the field for create (default 1) but label it "set automatically by Auto slots" and do not ask the admin to change it.

Section	Was	Now
5.4 Wallet buttons	Per-person buttons only.	New button Credit everyone (section 3.4).
6 Phase table, admin column	PREVIEW: open bidding. SOLVING: results, hints, credits.	PREVIEW: Slots card first. BIDDING: Bid coverage. SOLVING: Allocation page and Auto fill before anything else.
9 Edge cases	-	See section 5 below.

3. New admin screens

3.1 Slots card (PREVIEW)

- **Calls:** POST /api/admin/rounds/current/slots/auto?dryRun=true or false. dryRun is required; a missing value is a 400.
- **Shows:** participants counted (completed profiles only), problems, slots needed, slots assigned and slots per problem. Buttons: Preview (dry run) then Apply.
- **Rules:** PREVIEW only (INVALID_PHASE). VALIDATION_FAILED with details.participants when nobody has a profile yet, or details.problems when the round has fewer than 3 problems. Safe to press again; do it whenever someone new has signed in and always right before Open bidding.

3.2 Bid coverage (BIDDING)

- **Calls:** GET /api/admin/rounds/current/bid-coverage. Poll about every 15 s while the screen is open.
- **Shows:** every participant with bidCount and missingBids, fewest bids first. Highlight rows with missingBids greater than 0 so the admin can chase them before closing bidding.

3.3 Allocation page (SOLVING and RESULTS)

- **Calls:** GET .../allocation; POST .../auto-fill?dryRun=true|false; POST .../assignments with { userId, problemId }.
- **Shows:** "atTargetCount of participantCount at 3 problems"; a participant table (wins, missing, excess, unaffordableBids, coins) with people not at 3 first; a problem table (slots, sold, unsold, unaffordable, overCapacity).
- **Flow:** press Auto fill with a dry run, show the assignments and stillShort lists, confirm, apply. Then refetch. For every stillShort row offer "Give a problem": a picker of the round's problems, confirm popup with the price (the problem's basePrice), then POST .../assignments.
- **Rules:** Auto fill and assignments work in SOLVING only (INVALID_PHASE otherwise); the allocation view also in RESULTS. Auto fill is safe to press twice; two presses at the same moment may give a 409 CONCURRENT_UPDATE, so refetch. Manual assignment errors: CONFLICT (already holds it, or already has 3), INSUFFICIENT_BALANCE, PROBLEM_NOT_FOUND, USER_NOT_FOUND. overCapacity true in the response is allowed on purpose; show a small note.

3.4 Credit everyone (wallet buttons)

- **Call:** POST /api/admin/wallet/credit-all with { amount, reason, requestId }. amount 1 to 1,000,000; reason required, max 200, shown to every participant; requestId 8 to 100 characters.
- **requestId:** make a NEW UUID each time the confirm popup opens and reuse it if the same click is retried. The same id again returns 409 CONFLICT and credits nobody twice, so treat that 409 as "already applied", not as a failure.
- **Confirm popup:** amount, reason and the number of participants (count them from GET /api/admin/users). Run it after sign-ups close; people who sign up later do not get the coins.
- **Result:** { participants, amountEach, totalCredited }. Each participant gets their own CUSTOM ledger entry and a CUSTOM_ADJUSTMENT pop-up. A wrong run is undone entry by entry with the normal undo.

4. API additions

Call	Body / query	Success	Notes
POST /api/admin/rounds/current/slots/auto	?dryRun=true|false (required)	200 AutoSlotsResponse	PREVIEW only
GET /api/admin/rounds/current/bid-coverage	-	200 BidCoverageResponse	Any phase of the current round

Call	Body / query	Success	Notes
GET /api/admin/rounds/current/allocation	-	200 AllocationViewResponse	SOLVING or RESULTS
POST /api/admin/rounds/current/auto-fill	?dryRun=true|false (required)	200 AutoFillResponse	SOLVING only
POST /api/admin/rounds/current/assignments	ManualAssignmentRequest	200 ManualAssignmentResponse	SOLVING only
POST /api/admin/wallet/credit-all	CreditAllRequest	200 CreditAllResponse	Idempotent by requestId

Changed behaviour of existing calls: POST .../advance (leaving PREVIEW) can now answer INVALID_PHASE for too few problems or slots; PUT .../bid answers BID_EXCEEDS_BALANCE based on the 3 highest bids; POST .../settle now allocates the whole round at once with the cap of 3 wins per person.

5. Data types (add to section 11)

```
interface AutoSlotsResponse {
  dryRun: boolean;
  participants: number;         // completed profiles
  problemCount: number;
  slotsNeeded: number;          // 3 x participants
  slotsAssigned: number;        // equals slotsNeeded with at least 3 problems
  slotsPerProblem: { problemId: number; title: string; slots: number }[];
}
interface BidCoverageResponse {
  roundId: number; roundNumber: number; problemCount: number; winsPerRound: number;
  participants: { userId: number; name: string | null; hackerRankId: string | null;
                  bidCount: number; missingBids: number }[];     // fewest bids first
}
interface AllocationViewResponse {
  roundId: number; roundNumber: number; winsPerRound: number;
  participantCount: number; atTargetCount: number;
  participants: { userId: number; name: string | null; hackerRankId: string | null;
                  coins: number; wins: number; unaffordableBids: number;
                  missing: number; excess: number; atTarget: boolean }[];  // not at target first
  problems: { problemId: number; title: string; slots: number; sold: number;
              unaffordable: number; unsold: number; overCapacity: number }[];
}
interface AutoFillResponse {
  dryRun: boolean; winsPerRound: number;
  assignments: { userId: number; name: string | null; hackerRankId: string | null;
                 problemId: number; problemTitle: string; price: number }[];
  stillShort: { userId: number; name: string | null; hackerRankId: string | null;
                wins: number; missing: number }[];
}
interface ManualAssignmentRequest { userId: number; problemId: number }
interface ManualAssignmentResponse {
  userId: number; problemId: number; problemTitle: string;
  pricePaid: number; balanceAfter: number; winsNow: number;
  overCapacity: boolean;        // problem now has more winners than slots (allowed)
}
interface CreditAllRequest { amount: number; reason: string; requestId: string }
interface CreditAllResponse { participants: number; amountEach: number; totalCredited: number }
```

No new enums or event types. Given problems reuse LedgerType BID_SETTLEMENT and EventType BID_WON; the ledger reason says "automatically" or "by an admin" and actorName is the admin.

6. One round, updated (replaces the admin steps of section 8.3)

Phase	Admin does	Participant sees
PREVIEW	Add problems (slots left at default). Wait until sign-ups are done. Slots card: dry run, then Apply. Optionally set the deadline. Open bidding.	Problem cards with question previews. "Bidding opens soon".
BIDDING	Watch Bid coverage and chase people with missing bids. At the deadline, Close bidding.	Sealed bid screen with the "X of Y problems" banner.
SETTLEMENT	Run settlement.	"Bidding closed, results coming".
SOLVING	Open the Allocation page. Auto fill (dry run, then apply). Give a problem by hand to every stillShort row until everyone is at 3. Then hints, devil, credits as before. Show results.	Won and given problems with links; exactly 3 once the admin is done.
Between rounds	Credit everyone to refill the economy (everyone pays for 3 problems per round).	CUSTOM_ADJUSTMENT pop-up.

7. New edge cases (add to section 9)

Situation	What the server does	What the UI does
Someone signs in after the slots were set	Opening bidding is refused: slots below 3 x participants	Show the message; press Auto slots again, then open bidding
A participant never bids	They have 0 wins after settlement; Auto fill gives them 3 at the reserve price	Shown at the top of the allocation table until filled

Situation	What the server does	What the UI does
A participant cannot afford the reserve price, or only problems they already hold have free slots	Auto fill leaves them in stillShort	Show them with the reason unknown; use Manual assignment (INSUFFICIENT_BALANCE means credit them first)
Manual assignment to a full problem	Allowed; overCapacity is true	Small note "this problem now has more winners than slots"
Admin deducts coins between bidding and settlement	Winner who cannot pay becomes UNAFFORDABLE; the slot stays unsold	Auto fill can hand that slot to someone short
Credit everyone clicked twice	Second call answers 409 CONFLICT, nobody credited again	Treat as already applied; do not show an error toast
Auto fill pressed in two tabs	One waits for the other, or answers 409 CONCURRENT_UPDATE	Refetch the allocation view

8. Build order and tests (add to sections 12.3 and 12.4)

- **Build order:** after the admin round controls (step for the stepper and deadline), add the Slots card, then Bid coverage, then the Allocation page with Auto fill and Manual assignment, then Credit everyone. Update the bid screen total (top 3) and the Rules page text early, since participants see them first.
- **Mock data:** add one fixture each for AutoSlotsResponse, BidCoverageResponse, AllocationViewResponse (with at least one person at 0 and one at 3), AutoFillResponse (with a stillShort row), ManualAssignmentResponse (overCapacity true and false) and CreditAllResponse; add error fixtures for the new INVALID_PHASE messages and a 409 CONFLICT on credit-all.
- **Test plan:** bid screen disables Save only when the top 3 total exceeds coins; Slots card shows the dry run before applying; Allocation page puts people not at 3 first and refreshes after Auto fill; Manual assignment shows the confirm price and handles each error code; Credit everyone sends a fresh requestId per popup and treats 409 as already applied.
- **Definition of done:** the acceptance script in section 8 now includes: press Auto slots, check bid coverage, run settlement, Auto fill, finish stillShort by hand, confirm every participant shows exactly 3 problems, then Credit everyone.